Mathematics of Haptera Generation — Overview

A walk-through of `haptera_export.py`, layer by layer from the cone geometry up through volume convergence.

1. Cone geometry

The cone is a right circular cone of height H and base radius R with apex at $z = H$ and base at $z = 0$. The wall radius at height z is

$$r_{\text{wall}}(z) = \frac{R}{H}(H - z).$$

`cone_contains` requires the skeleton point (x, y, z) to satisfy $\sqrt{x^2 + y^2} \le r_{\text{wall}}(z) - r_{\text{tube}}$, so the *tube surface* (skeleton ± tube radius r) stays inside the wall.

`wall_proximity` returns the dimensionless ratio $\rho = \sqrt{x^2 + y^2}/(r_{\text{wall}}(z) - r)$, with $\rho = 1$ exactly grazing the wall.

`inward_direction` is the unit normal pointing *into* the cone wall. The cone surface is the level set $f(x, y, z) = \sqrt{x^2 + y^2} + \frac{R}{H}z - R = 0$; its gradient is $\left(\frac{x}{r}, \frac{y}{r}, \frac{R}{H}\right)$, so the inward unit normal is

$$\hat{n}_{\text{in}} = -\frac{1}{\sqrt{1 + (R/H)^2}}\left(\frac{x}{r}, \frac{y}{r}, \frac{R}{H}\right).$$

The non-zero z component is what makes branches slide *along* the slanted wall toward the apex, instead of simply being shoved horizontally.

2. Branching recursion

Each segment in `grow` is a 3D line segment of length $L =$ `SEG_LEN` in direction $\hat{d}$. When a segment terminates, k children are launched. Given the parent's unit direction $\hat{d} = (d_x, d_y, d_z)$, define $\phi = \arctan 2(d_y, d_x)$ — the parent's azimuth in the xy plane.

For each child $i \in \{0, \dots, k-1\}$ the script computes:

$$\theta_i = \frac{2\pi i}{k} + \phi + (\ell \cdot \tau) + \xi_1, \qquad s_i = 0.28 + 0.18\,\xi_2,$$

where ℓ is the depth level, $\tau =$ `TORSION` is per-level rotation, and ξ_1, ξ_2 are PRNG samples. The child direction (before normalization) is

$$\vec{d}_i = \hat{d} + \big(s_i \cos\theta_i,\ s_i \sin\theta_i,\ 0.3\,\xi_3 - 0.1\big).$$

So children inherit the parent direction plus a lateral kick of magnitude $\sim s_i$ at azimuth θ_i, with a small vertical jitter. The $\ell\tau$ term twists the *whole* fan of children at each depth — this prevents nested generations from stacking into the same plane.

The PRNG itself (`make_rng`) is a textbook linear congruential generator: $X_{n+1} = (1664525\,X_n + 1013904223) \bmod 2^{32}$, returned as $X_n / 2^{32}$.

3. Wall steering

`steer` blends the direction toward $\hat{n}_{\text{in}}$ once $\rho > \rho_0$ (= `STEER_ONSET`). Define the activation

$$t = \min\!\left(\frac{\rho - \rho_0}{1 - \rho_0},\ 1\right), \qquad w = S\,t^2,$$

where $S =$ `STEER_STRENGTH`. The new direction is

$$\hat{d}' = \mathrm{normalize}(\hat{d} + w\,\hat{n}_{\text{in}}).$$

The quadratic ramp t^2 gives a soft onset: small corrections far from the wall, strong corrections at the wall.

4. Helical lanes (intertwined mode)

In `helix_target_direction` each root i of N traces a helix from apex to base. Let $\zeta = (H - z)/H \in [0, 1]$ parametrize "fraction down from apex." The lane's helical waypoint at height z is

$$\phi_i(z) = \frac{2\pi i}{N} + 2\pi T\,\zeta, \qquad \rho_h(z) = \alpha\, r_{\text{wall}}(z),$$

with $T =$ `HELIX_TURNS`, $\alpha =$ `RADIAL_FRACTION`. The waypoint is

$$\vec{p}_i(z) = \big(\rho_h(z)\cos\phi_i,\ \rho_h(z)\sin\phi_i,\ z\big).$$

`grow` then blends this toward the current direction with weight `LANE_BIAS`:

$$\hat{d} \leftarrow \mathrm{normalize}\big(\hat{d} + \beta\,\widehat{\vec{p}_i(z') - \vec{o}}\big),$$

where $z' = o_z - L$ (one step ahead) and $\beta =$ `LANE_BIAS`.

5. Inter-haptera collision avoidance

`_seg_seg_closest` implements the standard segment–segment closest-point algorithm (Ericson §5.1.9). For segments $\vec{p}_1 \to \vec{p}_2$ and $\vec{p}_3 \to \vec{p}_4$, parametrize closest points as $\vec{c}_1 = \vec{p}_1 + s\vec{d}_1,\ \vec{c}_2 = \vec{p}_3 + t\vec{d}_2$ with $s, t \in [0,1]$. Setting $\partial\|\vec{c}_1 - \vec{c}_2\|^2/\partial s = \partial\|\cdot\|^2/\partial t = 0$ gives the linear system

$$\begin{pmatrix} \vec{d}_1\cdot\vec{d}_1 & -\vec{d}_1\cdot\vec{d}_2 \\ -\vec{d}_1\cdot\vec{d}_2 & \vec{d}_2\cdot\vec{d}_2 \end{pmatrix}\begin{pmatrix} s \\ t \end{pmatrix} = \begin{pmatrix} -\vec{d}_1\cdot\vec{r} \\ \vec{d}_2\cdot\vec{r} \end{pmatrix},$$

where $\vec{r} = \vec{p}_1 - \vec{p}_3$. The solution is clamped to $[0,1]^2$ (with the boundary cases handled per the algorithm).

When the gap between two tubes is $d < \rho_r(r_a + r_b)$, with $\rho_r =$ `REPEL_ONSET`, a quadratic repulsion is applied:

$$w = R_s\left(1 - \frac{d}{\rho_r(r_a + r_b)}\right)^2,$$

with $R_s =$ `REPEL_STRENGTH`. The repulsion direction $\hat{e} = (\vec{c}_{\text{new}} - \vec{c}_{\text{exist}})/\|\cdot\|$ is then projected onto the plane perpendicular to the forward direction $\hat{d}$:

$$\hat{e}_\perp = \mathbf{normalize}\Big(\hat{e} - (\hat{e}\cdot\hat{d})\hat{d}\Big).$$

This guarantees the deflection is *purely lateral* — the branch sidesteps without slowing or reversing. Up to `REPEL_RETRIES` iterations, then a binary search along the original direction finds the longest sub-segment that clears every tube.

6. Tube mesh — Rodrigues rotation

`tube_mesh` builds a z-aligned cylinder, then rotates it onto $\hat{d}$. With $\hat{z} = (0,0,1)$ and rotation axis $\hat{a} = \hat{z}\times\hat{d}/\|\hat{z}\times\hat{d}\|$, angle $\theta = \mathbf{arccos}(\hat{z}\cdot\hat{d})$, the Rodrigues matrix is

$$R = I + (\sin\theta)[\hat{a}]_\times + (1-\cos\theta)[\hat{a}]_\times^2,$$

written component-wise with $c = \cos\theta$, $s = \sin\theta$, $t = 1 - c$.

7. Volume convergence — the cubic model

The naive total cylinder volume is

$$V_{\text{naive}} = \sum_i \pi r_i^2 L_i \ \propto\ r^2$$

(if all radii scale by a factor f). At every junction, two cylinders' flat caps occupy overlapping space; `overlap_volume` approximates this overlap as a hemisphere per touching segment:

$$V_{\text{overlap}} = \sum_{\text{junctions}} \sum_{i} \tfrac{2}{3}\pi r_i^3 \;\propto\; r^3.$$

So if all radii are multiplied by f, the boolean-union volume should follow

$$V(f) \;\approx\; f^2\, V_{\text{naive}} - f^3\, V_{\text{overlap}}.$$

`cubic_correction` solves $V(f) = V_{\text{target}}$ for f by Newton iteration on $g(f) = V_{\text{naive}} f^2 - V_{\text{overlap}} f^3 - V_{\text{target}}$, with $g'(f) = 2V_{\text{naive}} f - 3V_{\text{overlap}} f^2$.

This cubic has a maximum where $g' = 0$: $f^* = \frac{2V_{\text{naive}}}{3V_{\text{overlap}}}$, giving

$$V_{\max} = \frac{4\, V_{\text{naive}}^3}{27\, V_{\text{overlap}}^2}.$$

If $V_{\text{target}} > V_{\max}$, no scaling can hit the target and the code falls back to the simple step $f = \sqrt{V_{\text{target}} / V_{\text{measured}}}$.

8. Initial radius and iterative root-finding

`build_segments` seeds the radii so the *naive* volume already approximates the target. With a perfect k-ary tree of depth D, the total path length per root is $(D+1)L$, so

$$V_{\text{naive}} \approx N\, \pi r_0^2\, (D+1) L \;\Rightarrow\; r_0 = \sqrt{\frac{V_{\text{base}}}{N \pi L (D+1)}}.$$

The iteration loop recomputes the actual boolean-union volume each iteration via manifold3d, then updates f via:

- **Bisection** once both an undershoot f_{lo} and overshoot f_{hi} have been seen (guaranteed convergence).
- **Damped cubic**: if the error grew vs. last iteration, $d \leftarrow \max(d/2, 0.05)$ and $f_{\text{applied}} = 1 + (f_{\text{cubic}} - 1)d$.
- **Cubic with relaxed damping** otherwise: $d \leftarrow \min(1.1\, d, 1)$.

The target each iteration is a fraction of the current convex hull volume:

$$V_{\text{target}} = (1 - \Phi)\, V_{\text{hull}}, \qquad \Phi = \text{TARGET\_INTERSTITIAL\_FRACTION},$$

with a small adjustment so the boss/hole volumes don't get charged to the haptera target:

$$V_{\text{haptera target}} = V_{\text{hull}} - \Phi V_{\text{hull}} - (V_{\text{m\_final}} - V_{\text{measured}}).$$

Convergence is declared when $|V_{\text{interstitial}} - \Phi V_{\text{hull}}| / (\Phi V_{\text{hull}}) \leq \text{TOLERANCE}$.

The headline ideas: (1) recursive branching with azimuthal fan + per-level torsion, (2) the cone is enforced as a soft boundary via gradient-normal steering with a quadratic ramp, (3) collision avoidance uses a clean segment-segment distance with a lateral-projection trick to keep deflections perpendicular to motion, and (4) the radius is solved by Newton iteration on a cubic model $V(f) = f^2 V_{\text{naive}} - f^3 V_{\text{overlap}}$ that captures both the cylinder mass and the junction over-counting.